

DAG-Based Workflow Orchestrator

Problem 2 | Node.js + TypeScript + PostgreSQL | Docker

10 phases | Per-phase AI prompts | Test matrix | Manual verification

This document is the phased build plan for Problem 2 (DAG-Based Workflow Orchestrator). Each phase specifies: what to build, which assignment goals it satisfies, the exact prompt to give to Claude Code / AI IDE, the automated tests to include, and the manual verification steps to perform before moving to the next phase. All phases are additive — each builds on the prior phase's codebase.

Key conventions across all phases: TypeScript strict mode | Fastify framework | PostgreSQL via pg (no ORM) | Docker Compose for local dev | .env for all config | Vitest for unit tests | ESLint + Prettier | Interfaces for all core contracts | Repository pattern for DB access | Dependency injection via constructor params (no DI container)

Phase Overview

#	Phase	Time	Rubric	Delivers
0	Scaffolding + Docker + DB	~2h	Infra	Containerized dev env, schema migrations, health check
1	Core data structures	~3h	25%	Min-heap, bounded semaphore, DFS cycle detector
2	Workflow submission API	~2h	30%	POST /workflows with validation, cycle detection, persistence
3	Ready-set scheduler + worker pool	~4h	25%+20%	Atomic pending_deps decrement, handler registry, semaphore-bounded execution
4	Retry + backoff + timeout	~2h	30%	Decorrelated jitter, Promise.race timeout, retry state machine
5	Crash recovery + heartbeat	~3h	20%	Strict 2-writes, heartbeat reaper, startup recovery, kill -9 test
6	Cron parser + scheduler	~3h	30%	5-field parser, IANA timezone, DST gap/fold, next-fire algorithm
7	Cancel + status API	~1.5h	30%	Cooperative cancel, GET /workflows/:id, workflow state machine
8	Integration tests	~3h	30%+20%	500-task diamond DAG, retry storm, kill -9 recovery
9	Docs + polish + Dockerfile	~1.5h	10%+15%	Design doc, architecture diagram, README, single-command setup

Total estimated time: ~25 hours across 3 days. Phases 0-4 on Day 1, Phases 5-7 on Day 2, Phases 8-9 on Day 3.

Phase 0: Project Scaffolding + Docker + Database

0.1 What we build

- Node.js + TypeScript project with strict tsconfig, ESLint, Prettier, Vitest
- Docker Compose with: app (Node.js), postgres (16-alpine), and a migration runner

- .env file with all config: PORT, DATABASE_URL, WORKER_COUNT, HEARTBEAT_INTERVAL_MS, REAPER_POLL_MS, LEASE_TIMEOUT_MS, CRON_TICK_MS
- Fastify server scaffold with /health endpoint
- Database migration system (raw SQL files in /migrations, applied on startup)
- Full Postgres schema from the system design v2 (workflows, tasks, task_dependencies, schedules)
- pg connection pool wrapper with typed query helper
- Logger (pino via Fastify's built-in)

0.2 Assignment boxes ticked

INFRA	Docker + one-command setup (docker compose up)
CODE QUALITY	TypeScript strict, ESLint, Prettier, consistent project structure
PERSISTENCE	PostgreSQL schema matches system design v2 exactly
DOCS	README with setup instructions

0.3 Project structure

```

dag-orchestrator/
├── docker-compose.yml
├── Dockerfile
├── .env.example
├── .env
├── package.json
├── tsconfig.json
├── vitest.config.ts
├── .eslintrc.json
├── .prettierrc
├── migrations/
│   ├── 001_create_workflows.sql
│   ├── 002_create_tasks.sql
│   ├── 003_create_task_dependencies.sql
│   └── 004_create_schedules.sql
├── src/
│   ├── index.ts # entry point
│   ├── server.ts # Fastify setup
│   ├── config.ts # env parsing + validation
│   ├── db/
│   │   ├── pool.ts # pg Pool wrapper
│   │   ├── migrate.ts # migration runner
│   │   └── interfaces/ # all type contracts
│   ├── task.ts
│   ├── workflow.ts
│   ├── schedule.ts
│   ├── routes/ # Fastify route handlers
│   ├── services/ # business logic
│   ├── repositories/ # DB access (repository pattern)
│   ├── data-structures/ # heap, semaphore, cycle-detect
│   ├── engine/ # scheduler loop, worker pool, reaper
│   ├── cron/ # cron parser + scheduler
│   ├── tests/
│   ├── unit/
│   ├── integration/
│   └── helpers/

```

0.4 Prompt for Claude Code

```

Create a Node.js + TypeScript project for a DAG-based workflow orchestrator.

```

PROJECT SETUP:

- Use Fastify as the HTTP framework with @fastify/sensible for error handling.
- TypeScript strict mode (strict: true, noUncheckedIndexedAccess: true in tsconfig).
- Vitest for testing. ESLint + Prettier. tsx for dev, tsup for build.
- pino logger (Fastify's built-in).

DOCKER:

- docker-compose.yml with: app (node:20-alpine, ports 3000), postgres (16-alpine, ports 5432).
- Dockerfile: multi-stage build (builder + runner), non-root user.
- .env.example with: PORT=3000, DATABASE_URL=postgres://daguser:dagpass@postgres:5432/dagdb, WORKER_COUNT=10, HEARTBEAT_INTERVAL_MS=5000, REAPER_POLL_MS=30000, LEASE_TIMEOUT_MS=15000, CRON_TICK_MS=10000
- The app container should wait for postgres to be healthy before starting.

DATABASE:

- Create a migration runner in src/db/migrate.ts that reads .sql files from /migrations in alphabetical order, tracks applied migrations in a migrations table.
- Create the following schema (SQL migration files):

```
CREATE TABLE workflows (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  status TEXT NOT NULL DEFAULT 'PENDING',  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE tasks (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  logical_id TEXT NOT NULL,  
  workflow_id UUID NOT NULL REFERENCES workflows(id) ON DELETE CASCADE,  
  handler_name TEXT NOT NULL,  
  input JSONB NOT NULL DEFAULT '{}',  
  status TEXT NOT NULL DEFAULT 'PENDING',  
  priority INT NOT NULL DEFAULT 0,  
  scheduled_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  submission_order BIGSERIAL,  
  attempts INT NOT NULL DEFAULT 0,  
  max_attempts INT NOT NULL DEFAULT 3,  
  backoff_base_ms INT NOT NULL DEFAULT 1000,  
  backoff_cap_ms INT NOT NULL DEFAULT 30000,  
  last_sleep_ms INT NOT NULL DEFAULT 0,  
  timeout_ms INT NOT NULL DEFAULT 30000,  
  pending_deps INT NOT NULL DEFAULT 0,  
  last_heartbeat_at TIMESTAMPTZ,  
  error TEXT,  
  completed_at TIMESTAMPTZ,  
  UNIQUE(workflow_id, logical_id)  
);  
  
CREATE TABLE task_dependencies (  
  task_id UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,  
  depends_on_task_id UUID NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,  
  PRIMARY KEY (task_id, depends_on_task_id)  
);  
  
CREATE TABLE schedules (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  cron_expression TEXT NOT NULL,  
  timezone TEXT NOT NULL DEFAULT 'UTC',  
  workflow_definition JSONB NOT NULL,  
  next_fire_at TIMESTAMPTZ NOT NULL,  
  enabled BOOLEAN NOT NULL DEFAULT TRUE,  
  last_fired_at TIMESTAMPTZ
```

```

);

Create indices: tasks(workflow_id, status), tasks(status, scheduled_at, priority),
partial index on tasks WHERE status='RUNNING', task_dependencies(depends_on_task_id),
schedules(next_fire_at) WHERE enabled=TRUE.

- Create src/db/pool.ts: a pg Pool wrapper that exports a typed query<T>(sql, params) helper.
- Create src/config.ts: parse and validate all env vars, export a typed Config object.
- Create interfaces in src/interfaces/ for Task, Workflow, Schedule, TaskStatus, WorkflowStatus.
- Create a GET /health route that checks the DB connection.

PATTERNS:
- Repository pattern: src/repositories/workflow.repository.ts, task.repository.ts, etc.
- All DB queries use parameterized queries (no string interpolation).
- Use interfaces for all core contracts.
- Constructor-based dependency injection (pass db pool, config, logger to constructors).

TESTS:
- Unit test for config.ts: valid env parsing, missing vars throw.
- Integration test: docker compose up, /health returns 200, migrations applied.

Do NOT implement any business logic yet. Just the scaffold, schema, and health check.

```

0.5 Manual verification

- Run **docker compose up --build** and confirm all containers start cleanly
- Hit **GET /health** — expect 200 with {status: 'ok', db: 'connected'}
- Connect to Postgres directly: **docker compose exec postgres psql -U daguser dagdb**
- Run **ldt** — confirm workflows, tasks, task_dependencies, schedules, migrations tables exist
- Run **ld tasks** — confirm logical_id, pending_deps, last_heartbeat_at columns present
- Run **npm test** — all unit tests green

Phase 1: Core Data Structures

1.1 What we build

- **MinHeap<T>** — generic binary min-heap with custom comparator. insert(), extractMin(), peek(), size(). Array-backed, sift-up and sift-down from scratch. No Array.sort().
- **BoundedSemaphore** — integer counter + Promise-based waiter queue. acquire(), release(). Configurable limit.
- **detectCycle()** — DFS three-coloring cycle detector. Takes an adjacency list, returns null (no cycle) or the exact cycle path as string[].

1.2 Assignment boxes ticked

DATA STRUCT 25%	Min-heap with O(log n) insert/extract, compound key, sift ops from scratch
DATA STRUCT 25%	Bounded semaphore with counter + waiter queue (no p-limit/bottleneck)
CORRECTNESS 30%	DFS three-coloring returns exact cycle path for adversarial DAGs

1.3 Prompt for Claude Code

```

Implement three core data structures in src/data-structures/. Each must be a standalone
class/function with zero external dependencies. Follow Node.js best practices: generic types,
clean interfaces, JSDoc comments on public methods.

1. src/data-structures/min-heap.ts
- Generic class MinHeap<T> with a comparator: (a: T, b: T) => number.
- Methods: insert(item: T), extractMin(): T | undefined, peek(): T | undefined, size(): number.

```

- Backed by a plain array. Implement siftUp() and siftDown() with index arithmetic.
- DO NOT use Array.sort() anywhere. This is explicitly forbidden by the spec.
- The heap is keyed on compound key: (-priority, scheduledAt, submissionOrder).
- Export a TaskHeapEntry interface: { taskId: string, priority: number, scheduledAt: number, submissionOrder: bigint }.
- Export a createTaskComparator() function returning the right comparator.

2. src/data-structures/bounded-semaphore.ts

- Class BoundedSemaphore with constructor(limit: number).
- acquire(): Promise<void> – if count > 0, decrement and return immediately. Otherwise, push a resolver onto a waiter queue and await the promise.
- release(): void – if waiters exist, shift and resolve one. Otherwise, increment count.
- availablePermits(): number for observability.
- No external packages (no p-limit, async-pool, bottleneck). Spec forbids them.

3. src/data-structures/cycle-detector.ts

- Function detectCycle(adjacency: Map<string, string[]>): string[] | null
- DFS three-coloring: WHITE (unvisited), GRAY (in stack), BLACK (done).
- On back-edge to GRAY node, extract the cycle path from the DFS stack and return it.
- Handle: self-loops, diamonds, disconnected components (run DFS from all WHITE nodes).
- Return null if no cycle found.

TESTS (tests/unit/data-structures/):

min-heap.test.ts:

- Insert 1000 random items, extractMin should return them in sorted order.
- Priority tie-breaking: same priority -> earlier scheduledAt wins.
- ScheduledAt tie-breaking: same priority + scheduledAt -> smaller submissionOrder wins.
- Extract from empty heap returns undefined.
- Peek does not remove the element.
- Insert + extract interleaved maintains heap invariant.
- 10K entries stress test: all extracted in correct order.

bounded-semaphore.test.ts:

- Semaphore(3): acquire 3 times immediately, 4th blocks until release.
- Release unblocks oldest waiter first (FIFO).
- availablePermits() reflects current state.
- Rapid acquire/release cycle (1000 iterations): no deadlock, no drift.
- Concurrent scenario: 10 tasks with semaphore(3), max 3 running simultaneously.

cycle-detector.test.ts:

- Self-loop: A -> A. Returns ['A', 'A'].
- Simple cycle: A -> B -> C -> A. Returns path.
- Diamond: A -> B, A -> C, B -> D, C -> D. Returns null (valid DAG).
- Disconnected components: two separate subgraphs, one with cycle. Detects the cycle.
- Long chain (100 nodes) with back-edge at end. Returns full cycle path.
- Empty graph. Returns null.
- Single node, no edges. Returns null.
- Multiple cycles: returns any one of them.

1.4 Manual verification

- Run **npm test -- tests/unit/data-structures/** — all tests green
- Check heap: console.log a trace of sift operations on 5 inserts to verify correct index swaps
- Check semaphore: add a console.log in acquire/release, run the concurrent test, confirm max 3 in parallel
- Check cycle detector: log the color transitions for a 4-node diamond DAG to confirm no false positive

Phase 2: Workflow Submission API

2.1 What we build

- **POST /workflows** — accepts a workflow definition (list of tasks with dependsOn), validates, persists, returns workflowId
- **Submission flow:** validate JSON shape → validate handler names against registry → run cycle detection → compute pending_deps per task → BEGIN tx → insert workflow + tasks + dependencies → mark zero-dep tasks READY → workflow RUNNING → COMMIT
- **HandlerRegistry** — in-process Map<string, HandlerFn> with register() and has() methods
- **WorkflowService** — orchestrates submission logic, depends on repositories + cycle detector + handler registry

2.2 Assignment boxes ticked

CORRECTNESS 30%	Cycle detection rejects adversarial DAGs with exact cycle path in 422 error
CORRECTNESS 30%	pending_deps correctly computed from dependsOn; zero-dep tasks marked READY
CODE QUALITY 15%	Service layer, repository pattern, typed interfaces, validation

2.3 Prompt for Claude Code

```
Build the POST /workflows endpoint. This is the submission path for DAG workflows.

HANDLER REGISTRY (src/engine/handler-registry.ts):
- Class HandlerRegistry with register(name: string, fn: HandlerFn) and get(name): HandlerFn.
- HandlerFn = (input: unknown, ctx: HandlerContext) => Promise<void>.
- HandlerContext = { idempotencyKey: string, signal: AbortSignal, workflowId: string,
taskId: string, attempt: number }.
- has(name): boolean for validation.

WORKFLOW SERVICE (src/services/workflow.service.ts):
- Constructor takes: db pool, taskRepo, workflowRepo, handlerRegistry, logger.
- Method: submitWorkflow(definition: WorkflowDefinition): Promise<{ workflowId: string }>
- Submission flow (single Postgres transaction):
1. Validate every task.handler exists in the registry. If not, throw 422.
2. Build adjacency map from dependsOn. Run detectCycle(). If cycle, throw 422 with path.
3. Compute pending_deps for each task = dependsOn.length.
4. Insert workflow row (status='RUNNING').
5. Insert all task rows. Use logical_id for the caller-supplied task id.
Map logical_id -> internal UUID during insert.
6. Insert task_dependencies rows (resolve logical_id to UUID for FK refs).
7. UPDATE tasks SET status='READY' WHERE pending_deps=0 AND workflow_id=$1.
8. COMMIT and return { workflowId }.

WORKFLOW DEFINITION interface (src/interfaces/workflow.ts):
interface TaskDefinition {
id: string; // caller-supplied logical ID
handler: string; // registered handler name
input: unknown; // arbitrary JSON
dependsOn: string[]; // logical IDs of dependencies
retryPolicy?: { maxAttempts: number; backoffBase: number; backoffCap: number };
timeoutMs?: number;
priority?: number;
}
interface WorkflowDefinition { tasks: TaskDefinition[] }

ROUTE (src/routes/workflow.routes.ts):
- POST /workflows: parse body, call workflowService.submitWorkflow(), return 201.
- Use Fastify schema validation for the request body.

REPOSITORIES:
- WorkflowRepository: insertWorkflow(tx, status) -> workflowId
- TaskRepository: bulkInsertTasks(tx, workflowId, tasks[]), insertDependencies(tx, deps[]),
markReadyTasks(tx, workflowId), getTasksByWorkflowId(workflowId)
```

```

Register a few test handlers at startup: 'noop' (does nothing), 'echo' (returns input),
'fail-once' (fails on first attempt, succeeds on second), 'slow' (sleeps for input.ms).

TESTS (tests/unit/services/workflow-submission.test.ts):
- Valid 3-task chain A -> B -> C: returns workflowId, tasks A is READY, B and C are PENDING.
- Diamond DAG A -> B,C -> D: A is READY, D has pending_deps=2.
- Cycle A -> B -> A: returns 422 with cycle path in error message.
- Self-loop A -> A: returns 422.
- Unknown handler: returns 422 with handler name in error.
- All tasks have no dependencies: all marked READY immediately.
- Empty tasks array: returns 422.
- Duplicate task IDs: returns 422.

INTEGRATION TESTS (tests/integration/submission.test.ts):
- POST /workflows with valid 5-task DAG: 201 response with workflowId.
- POST /workflows with cycle: 422 response.
- GET /health still works after submission.
- Verify tasks in DB via direct SQL query match expected states.

```

2.4 Manual verification

- Use curl/httpie to POST a valid 3-task chain workflow. Verify 201 response.
- Query DB directly: `SELECT logical_id, status, pending_deps FROM tasks WHERE workflow_id = '...'`
- Confirm first task is READY (pending_deps=0), others PENDING
- POST a workflow with cycle (A->B->C->A). Confirm 422 with cycle path in the error body.
- POST with unknown handler name. Confirm 422 with clear error.

Phase 3: Ready-Set Scheduler + Worker Pool

3.1 What we build

- **SchedulerLoop** — tick-based loop that loads READY tasks from DB into the min-heap, dequeues and dispatches to workers
- **WorkerPool** — uses BoundedSemaphore to limit concurrency, executes handlers via HandlerRegistry
- **Worker execution flow:** extractMin from heap → CAS guard (UPDATE WHERE status='READY') → mark RUNNING (Write 1) → start heartbeat → execute handler with timeout → persist outcome (Write 2) → decrement children's pending_deps atomically → enqueue newly READY children
- **Ready-set decrement:** UPDATE tasks SET pending_deps=pending_deps-1 WHERE id=\$1 AND status='PENDING' RETURNING id, pending_deps, priority, scheduled_at, submission_order

3.2 Assignment boxes ticked

DATA STRUCT 25%	Min-heap used as scheduling queue; CAS guard for multi-process future
CONCURRENCY 20%	Bounded semaphore limits concurrent workers; atomic ready-set update
CORRECTNESS 30%	No double-enqueue: only the worker that sees pending_deps=0 in RETURNING enqueues
CODE QUALITY 15%	Clean separation: SchedulerLoop, WorkerPool, TaskExecutor

3.3 Prompt for Claude Code

```

Build the scheduler loop and worker pool. This is the execution engine core.

SCHEDULER LOOP (src/engine/scheduler-loop.ts):
- On startup: SELECT all tasks WHERE status='READY' AND scheduled_at <= NOW().

```

Insert each into the min-heap.

- Tick every 100ms (configurable): if heap has items and semaphore has permits, `extractMin()` and dispatch to worker pool.
- The heap is the authoritative queue in single-process mode. DO NOT use `SELECT FOR UPDATE SKIP LOCKED` for task claiming – that's for multi-process (see §12 of design).
- Provide `start()` and `stop()` methods for lifecycle management.

WORKER POOL (`src/engine/worker-pool.ts`):

- Constructor takes: semaphore, handlerRegistry, taskRepo, db, config, logger.
- Method: `executeTask(heapEntry: TaskHeapEntry): Promise<void>`
 1. await semaphore.acquire()
 2. CAS guard: `UPDATE tasks SET status='RUNNING', attempts=attempts+1, last_heartbeat_at=NOW() WHERE id=$1 AND status='READY' RETURNING attempts`. If `rowCount=0`, release semaphore and return (stale pop).
 3. Start heartbeat interval: `UPDATE tasks SET last_heartbeat_at=NOW() WHERE id=$1 every HEARTBEAT_INTERVAL_MS`.
 4. Build stable idempotency key: ``${workflowId}:${task.logicalId}``.
 5. Create AbortController + setTimeout for task.timeoutMs.
 6. Execute handler via `Promise.race([handler(input, ctx), timeoutPromise])`.
 7. On success: `persistOutcome(task, 'COMPLETED')`.
 8. On failure: `persistOutcome(task, 'FAILED', error)`.
 9. `clearTimeout, clearInterval(heartbeat), semaphore.release()` in finally block.

TASK COMPLETION (in `worker-pool.ts` or `src/engine/task-completer.ts`):

- After a task completes or fails terminally:
 1. Query `task_dependencies` for children of this task.
 2. For each child: `UPDATE tasks SET pending_deps=pending_deps-1 WHERE id=$1 AND status='PENDING' RETURNING id, pending_deps, priority, scheduled_at, submission_order`.
 3. If returned `pending_deps === 0`: `UPDATE tasks SET status='READY', scheduled_at=NOW() WHERE id=$1`. Insert into heap.
- This is the atomic ready-set. The `UPDATE...RETURNING` prevents TOCTOU race.

PERSIST OUTCOME (`src/engine/task-completer.ts`):

- On COMPLETED: `UPDATE tasks SET status='COMPLETED', completed_at=NOW() WHERE id=$1`. Then trigger ready-set for children.
 - Then check if all tasks in workflow are terminal -> workflow COMPLETED or FAILED.
 - On FAILED (terminal – attempts >= max_attempts):
`UPDATE tasks SET status='FAILED', error=$2, completed_at=NOW() WHERE id=$1`. Then check workflow status.
 - On FAILED (retryable – attempts < max_attempts): handled in Phase 4.
- For now, just mark FAILED terminally so we can test the happy path.

WORKFLOW COMPLETION CHECK:

- After each task reaches terminal state, query:
`SELECT COUNT(*) FILTER (WHERE status NOT IN ('COMPLETED','FAILED','CANCELLED')) FROM tasks WHERE workflow_id=$1`.
- If 0: workflow is terminal. If any FAILED, workflow status='FAILED'. Else 'COMPLETED'.

TESTS (`tests/unit/engine/`):

`scheduler-loop.test.ts`:

- Loads READY tasks from DB into heap on startup.
- Dispatches tasks from heap to worker pool.
- Does not dispatch if semaphore is full.

`worker-pool.test.ts`:

- Executes a 'noop' handler and marks task COMPLETED.
- CAS guard: if task already claimed (`status != READY`), skips execution.
- Heartbeat: `last_heartbeat_at` updates during execution.
- Handler timeout: slow handler past `timeoutMs` is rejected, task marked FAILED.

`task-completer.test.ts`:

- 3-task chain A -> B -> C: completing A makes B READY (`pending_deps 1->0`).
- Diamond: completing B (one of D's two parents) decrements D from 2 to 1.


```

D stays PENDING. Completing C decrements D from 1 to 0, D becomes READY.
- Concurrent parent completion (simulate): two UPDATE RETURNING calls on same child.
Only one sees pending_deps=0.

INTEGRATION TEST (tests/integration/execution.test.ts):
- Submit a 5-task chain. Wait for workflow to reach COMPLETED. All tasks COMPLETED.
- Submit a diamond DAG (A -> B,C -> D). All 4 tasks COMPLETED in correct order.
- Submit with 'fail-once' handler (for now, will fail terminally). Workflow FAILED.

```

3.4 Manual verification

- Submit a 3-task chain via curl. Watch logs for task execution order (A, then B, then C)
- Query DB: SELECT logical_id, status, attempts FROM tasks ORDER BY completed_at
- Submit a diamond DAG. Confirm D only executes after both B and C complete
- Set WORKER_COUNT=1 in .env. Submit a 3-task parallel workflow. Confirm tasks run one at a time

Phase 4: Retry + Backoff + Timeout

4.1 What we build

- **Decorrelated jitter backoff:** $\text{sleep} = \min(\text{cap}, \text{random}(\text{base}, \text{prevSleep} * 3))$. last_sleep_ms persisted in DB.
- **Retry state machine:** on failure with attempts < max_attempts, transition back to PENDING with future scheduled_at. Scheduler only dequeues when scheduled_at <= now.
- **Hard timeout:** Promise.race with AbortSignal (already scaffolded in Phase 3, now tested)
- **Canonical attempt counting:** attempts incremented once at RUNNING entry. Not in failure path. Not in reaper.

4.2 Assignment boxes ticked

CORRECTNESS 30%	Backoff math: computed retry timestamps match decorrelated jitter formula exactly
CORRECTNESS 30%	Retry storm: 100 tasks failing 5 times each, spread across time by jitter
DATA STRUCT 25%	Scheduler respects scheduled_at: does not dequeue before time

4.3 Prompt for Claude Code

```

Add retry + backoff + timeout mechanics to the execution engine.

BACKOFF CALCULATOR (src/engine/backoff.ts):
- Function: computeNextSleep(backoffBase: number, backoffCap: number, lastSleep: number): number
- Formula: const prevSleep = lastSleep || backoffBase;
const raw = Math.random() * (prevSleep * 3 - backoffBase) + backoffBase;
return Math.min(backoffCap, raw);
- This is decorrelated jitter (AWS architecture blog). Self-adapting: short prev sleep
-> narrow range. Long prev sleep -> wide range.

RETRY STATE MACHINE (update src/engine/task-completer.ts):
- On handler failure, check: task.attempts < task.maxAttempts?
YES (retryable):
const sleep = computeNextSleep(task.backoffBase, task.backoffCap, task.lastSleep);
UPDATE tasks SET status='PENDING', scheduled_at=NOW() + sleep ms,
last_sleep_ms=sleep WHERE id=$1.
DO NOT increment attempts here (already incremented at RUNNING entry).
DO NOT insert into heap yet (scheduled_at is in the future).
NO (terminal failure):
UPDATE tasks SET status='FAILED', error=$2, completed_at=NOW() WHERE id=$1.

```

```

DO NOT trigger ready-set for children (failed parent = children never run).
Check workflow completion.

SCHEDULER LOOP UPDATE:
- On each tick, also poll DB for PENDING tasks with scheduled_at <= NOW() that are
not yet in the heap. This catches retry-eligible tasks whose backoff has elapsed.
SELECT * FROM tasks WHERE status='PENDING' AND pending_deps=0
AND scheduled_at <= NOW() AND id NOT IN (heap task IDs).
Insert found tasks into heap.

CANONICAL ATTEMPT COUNTING RULE:
- attempts is incremented ONLY in the CAS guard: UPDATE SET attempts=attempts+1
WHERE status='READY'. This happened in Phase 3. Verify it is not incremented elsewhere.
- The reaper (Phase 5) does NOT increment attempts.
- The retry path does NOT increment attempts.

TESTS (tests/unit/engine/backoff.test.ts):
- computeNextSleep with base=1000, cap=30000, lastSleep=0: result in [1000, 1000].
- computeNextSleep with base=1000, cap=30000, lastSleep=1000: result in [1000, 3000].
- computeNextSleep with base=1000, cap=30000, lastSleep=10000: result in [1000, 30000].
- Cap is respected: never returns > cap.
- Run 1000 times: verify distribution spread (no clustering at 0 or cap).

TESTS (tests/unit/engine/retry.test.ts):
- Task with maxAttempts=3: fails, transitions to PENDING with future scheduled_at.
Fails again (attempt 2), scheduled again. Fails again (attempt 3), terminal FAILED.
- Verify last_sleep_ms is updated in DB after each retry schedule.
- Verify scheduled_at is NOW() + computed sleep (within 100ms tolerance).
- Verify attempts is NOT incremented in the retry path.

TESTS (tests/unit/engine/timeout.test.ts):
- Task with timeoutMs=100, handler sleeps 500ms: task fails with 'timeout' error.
- Task with timeoutMs=5000, handler completes in 10ms: task succeeds.
- AbortSignal is set when timeout fires (verify signal.aborted === true).

INTEGRATION TEST (tests/integration/retry.test.ts):
- Submit workflow with 'fail-n-times' handler (fails first N-1 times, succeeds on Nth).
maxAttempts = 5, handler fails 3 times. Workflow eventually COMPLETED.
- Submit with maxAttempts=1 and failing handler. Task goes directly to FAILED.

```

4.4 Manual verification

- Submit workflow with 'fail-once' handler (maxAttempts=3). Watch logs: first attempt fails, second succeeds
- Query DB: SELECT attempts, last_sleep_ms, scheduled_at FROM tasks — confirm backoff values
- Submit with timeoutMs=100 and 'slow' handler (input.ms=5000). Confirm timeout failure in logs

Phase 5: Crash Recovery + Heartbeat Reaper

5.1 What we build

- **Heartbeat:** already implemented in Phase 3 worker loop. Verify interval update.
- **Reaper loop:** runs at startup and every REAPER_POLL_MS. Single atomic UPDATE...RETURNING to reclaim stale RUNNING tasks.
- **Startup recovery sequence:** run reaper → load READY tasks into heap → resume scheduler → fire overdue cron schedules (Phase 6)
- **kill -9 test:** spawn process, submit workflow, kill mid-execution, restart, assert recovery

5.2 Assignment boxes ticked

DURABILITY 20%	Two writes per transition: RUNNING before handler, COMPLETED/FAILED after
-----------------------	---

DURABILITY 20%	Reaper reclaims stale RUNNING tasks via atomic UPDATE (no TOCTOU race)
DURABILITY 20%	kill -9 test proves no acknowledged task is lost
CORRECTNESS 30%	Reaper respects retry policy: reclaimed tasks go PENDING (retryable) or FAILED (exhausted)

5.3 Prompt for Claude Code

```
Build the crash recovery subsystem: heartbeat reaper and startup recovery.

REAPER (src/engine/reaper.ts):
- Class Reaper with constructor(db, heap, config, logger).
- Method: reclaimStaleTasks(): Promise<number> (returns count of reclaimed tasks).
- Uses a SINGLE atomic UPDATE...RETURNING query (no SELECT then UPDATE):

WITH reclaimed AS (
UPDATE tasks
SET status = CASE
WHEN attempts >= max_attempts THEN 'FAILED'
ELSE 'PENDING'
END,
scheduled_at = CASE
WHEN attempts >= max_attempts THEN NOW()
ELSE NOW() + (LEAST(backoff_cap_ms,
FLOOR(random() * (last_sleep_ms * 3 - backoff_base_ms)
+ backoff_base_ms)) || ' milliseconds')::interval
END,
last_sleep_ms = CASE
WHEN attempts >= max_attempts THEN last_sleep_ms
ELSE LEAST(backoff_cap_ms,
FLOOR(random() * (last_sleep_ms * 3 - backoff_base_ms)
+ backoff_base_ms))
END
WHERE status = 'RUNNING'
AND last_heartbeat_at < NOW() - INTERVAL '${LEASE_TIMEOUT_MS} milliseconds'
RETURNING *
)
SELECT * FROM reclaimed;

IMPORTANT: Do NOT increment attempts. It was already incremented when the task first
entered RUNNING. Incrementing again would cause double-counting and break max_attempts.
- For reclaimed tasks that became PENDING with scheduled_at <= NOW(), insert into heap.
- For reclaimed tasks that became FAILED, check workflow completion.
- Method: startPolling(): void – setInterval(reclaimStaleTasks, REAPER_POLL_MS).
- Method: stop(): void – clearInterval.

STARTUP RECOVERY (update src/index.ts or src/engine/engine.ts):
- On app start, before accepting HTTP traffic:
1. Run migrations.
2. Run reaper.reclaimStaleTasks() once.
3. Load all READY tasks (scheduled_at <= NOW()) into heap.
4. Start scheduler loop.
5. Start reaper polling.
6. Start Fastify listener.

TESTS (tests/unit/engine/reaper.test.ts):
- Task stuck in RUNNING with stale heartbeat (> LEASE_TIMEOUT_MS ago):
reaper reclaims it to PENDING with future scheduled_at.
- Task stuck in RUNNING with attempts >= max_attempts: reaper marks FAILED.
- Task in RUNNING with fresh heartbeat (< LEASE_TIMEOUT_MS): NOT reclaimed.
- Two stale tasks: both reclaimed in single query.
- Verify attempts is NOT incremented by reaper.
```

```

- Verify last_sleep_ms is updated with decorrelated jitter.
INTEGRATION TEST (tests/integration/crash-recovery.test.ts):
- kill -9 test:
1. Start the app as a child process.
2. Submit a 10-task chain with 'slow' handler (500ms each).
3. Wait until 3 tasks are RUNNING (poll GET /workflows/:id).
4. Kill the process with SIGKILL (kill -9).
5. Start a new instance.
6. Wait for workflow to reach terminal state (poll with 60s timeout).
7. Assert: all 10 tasks are COMPLETED or FAILED. None stuck in RUNNING.
8. Assert: no task has attempts > max_attempts.
This test is a REQUIRED DELIVERABLE. Make it robust: use proper timeouts,
clear Postgres state between runs, handle port conflicts.

```

5.4 Manual verification

- Submit a slow workflow (10 tasks, 2s each). While running, kill the process (kill -9 PID)
- Restart the app. Watch logs for 'Reclaimed N stale tasks'
- Query DB: confirm no tasks stuck in RUNNING after recovery
- Run the kill -9 integration test: **npm test -- tests/integration/crash-recovery.test.ts**

Phase 6: Cron Parser + Scheduler

6.1 What we build

- **Cron parser:** parse 5-field expressions (minute, hour, DOM, month, DOW) supporting *, */n, ranges, lists
- **next-fire algorithm:** minute-by-minute scan with DST-aware resolution via Intl.DateTimeFormat
- **DST handling:** skip gap (spring forward), fire once on fold (fall back, first occurrence)
- **POST /schedules:** register a cron schedule with inline workflow definition and IANA timezone
- **Cron scheduler loop:** polls schedules table for due fires, clones workflow definition, submits via submitWorkflow()

6.2 Assignment boxes ticked

CORRECTNESS 30%	Cron parser handles all required syntax: *, */n, ranges, lists
CORRECTNESS 30%	DST gap/fold handling: spec explicitly evaluates this
CORRECTNESS 30%	0 0 29 2 * correctly fires on Feb 29 (leap year only)
DATA STRUCT 25%	Cron parser implemented from scratch (no node-cron/croner)

6.3 Prompt for Claude Code

```

Build the cron parser and cron scheduler. Implement from scratch – no cron libraries.

CRON_PARSER (src/cron/parser.ts):
- Function: parseCronExpression(expr: string): CronExpression
- CronExpression = { minutes: Set<number>, hours: Set<number>,
daysOfMonth: Set<number>, months: Set<number>, daysOfWeek: Set<number> }
- Parse each of the 5 fields. Support:
* (wildcard) -> all values in range
*/n (step) -> every nth value starting from range start
n (single value)
n-m (range, inclusive)
n,m,k (list)

```

Combinations: 1-5,10,15 = {1,2,3,4,5,10,15}

- Validate ranges: minute 0-59, hour 0-23, DOM 1-31, month 1-12, DOW 0-6 (0=Sun).
- Throw on invalid expressions with descriptive error message.

NEXT FIRE (src/cron/next-fire.ts):

- Function: nextFire(expr: CronExpression, after: Date, timezone: string): Date
- Algorithm: advance from 'after' one minute at a time. For each candidate minute, convert to local wall-clock time in the given IANA timezone using Intl.DateTimeFormat. Check if minute, hour, DOM, month, DOW all match the expression. If match, resolve DST ambiguity and return the UTC Date.
- Max search: 366 * 24 * 60 iterations (1 year). Throw if no match found.
- DST gap (spring forward, e.g. 2:00AM skipped): if computed local time falls in gap, skip this candidate. The next valid minute will be found.
- DST fold (fall back, e.g. 2:00AM repeated): fire on the first occurrence only (use the standard-time offset, not daylight-time offset).
- Use Intl.DateTimeFormat with timeZone option to resolve local time fields.

CRON SCHEDULER (src/cron/cron-scheduler.ts):

- Class CronScheduler with constructor(db, workflowService, config, logger).
- Method: tick(): Poll schedules WHERE enabled=TRUE AND next_fire_at <= NOW(). For each due schedule: deep-clone workflow_definition, submit via workflowService, compute next_fire_at, update schedule row.
- Method: startPolling(): setInterval(tick, CRON_TICK_MS).
- Method: stop(): clearInterval.

POST /schedules ROUTE:

- Accepts: { cronExpression, timezone?, workflowDefinition }.
- Parse cron expression (validate). Compute initial next_fire_at.
- Insert schedule row. Return { scheduleId, nextFireAt }.

TESTS (tests/unit/cron/parser.test.ts) — at least 15 expressions:

1. '*' * * * * -> every minute (all sets full)
2. '*/15' * * * * * -> minutes {0,15,30,45}
3. '0' * * * * * -> minute 0 only
4. '0 9' * * * * * -> 09:00 daily
5. '0 9' * * * 1-5 -> 09:00 weekdays only
6. '30 8,12,17' * * * * * -> 08:30, 12:30, 17:30
7. '0 0 1' * * * * * -> midnight on 1st of each month
8. '0 0 29 2' * * * * * -> midnight Feb 29 (leap year)
9. '0 0 1 1' * * * * * -> midnight Jan 1
10. '*/5' * * * * * -> every 5 minutes
11. '0 0 * * 0' -> midnight every Sunday
12. '0 6-18' * * * 1-5 -> every hour 6AM-6PM weekdays
13. '0 0 15,30' * * * * * -> midnight on 15th and 30th
14. '59 23 31 12' * * * * * -> 23:59 Dec 31
15. '0,30' * * * * * -> every half hour
16. Invalid: '60' * * * * * -> throws error
17. Invalid: '* * * * * *' (6 fields) -> throws error

TESTS (tests/unit/cron/next-fire.test.ts):

- '*/15' * * * * * after 2024-01-01T00:00:00Z: next = 2024-01-01T00:15:00Z.
- '0 0 29 2' * * * * * after 2024-03-01: next = 2028-02-29T00:00:00Z.
- '0 9' * * * 1-5 after Friday 17:00: next = Monday 09:00.
- DST spring forward: '0 2' * * * * * in America/New_York on March 10, 2024. 2:00 AM doesn't exist. Next fire should be 3:00 AM local (07:00 UTC).
- DST fall back: '0 1' * * * * * in America/New_York on Nov 3, 2024. 1:00 AM happens twice. Should fire on first occurrence (05:00 UTC).
- '* * * * *' crosses midnight: correct date rollover.

6.4 Manual verification

- Run cron parser unit tests: **npm test -- tests/unit/cron/** — all 17+ tests green

- POST a schedule via curl: cronExpression `*/1 * * * *`, timezone UTC. Confirm nextFireAt in response is ~1 min from now.
- Wait 1-2 minutes, query DB: `SELECT * FROM schedules` — confirm last_fired_at is populated, next_fire_at advanced
- Confirm a workflow was auto-created by the cron fire: `SELECT * FROM workflows ORDER BY created_at DESC`

Phase 7: Cancel + Status API

7.1 What we build

- **POST /workflows/:id/cancel** — cooperative cancel: `RUNNING` → `CANCELLING` → `CANCELLED`
- **GET /workflows/:id** — returns workflow status + per-task statuses + overall state
- **Workflow state machine enforcement** — only valid transitions allowed
- **Cancel logic:** mark workflow `CANCELLING`; workers check before dequeuing; running tasks drain; dependents `CANCELLED`

7.2 Assignment boxes ticked

CORRECTNESS 30%	Cancel: no orphaned in-flight tasks; running drain, dependents skipped
CODE QUALITY 15%	Idempotent cancel (re-cancel returns 200 no-op)
DOCS 10%	GET /workflows/:id returns clear, complete status

7.3 Prompt for Claude Code

```
Build cancel and status APIs. Cooperative cancel model.

GET /workflows/:id ROUTE:
- Query workflow row + all tasks for that workflow.
- Return: { id, status, createdAt, updatedAt, tasks: [{ logicalId, status, handler, attempts, maxAttempts, error, completedAt, ... }] }
- If workflow not found, return 404.

POST /workflows/:id/cancel ROUTE:
- Query current workflow status.
- If COMPLETED or FAILED: return 409 'Workflow already terminal'.
- If CANCELLED or CANCELLING: return 200 (idempotent no-op).
- If RUNNING: UPDATE workflows SET status='CANCELLING', updated_at=NOW() WHERE id=$1.
- Cancel non-started tasks: UPDATE tasks SET status='CANCELLED', completed_at=NOW() WHERE workflow_id=$1 AND status IN ('PENDING', 'READY').
- Running tasks: let them drain. Workers check workflow status before execution.

WORKER POOL UPDATE:
- Before executing a handler, check: SELECT status FROM workflows WHERE id=$1. If CANCELLING, mark task CANCELLED and return without executing.
- After a running task completes in a CANCELLING workflow, mark dependents CANCELLED.
- After all tasks are terminal and workflow is CANCELLING: UPDATE workflows SET status='CANCELLED' WHERE id=$1.

TESTS:
- Cancel a RUNNING workflow: tasks in READY/PENDING become CANCELLED. Running tasks complete normally. Dependents never enqueued.
- Cancel already-cancelled workflow: 200, no-op.
- Cancel COMPLETED workflow: 409.
- GET /workflows/:id during execution: correct per-task statuses.
- GET /workflows/:id for completed workflow: all statuses accurate.
- GET /workflows/:id for non-existent ID: 404.
```

7.4 Manual verification

- Submit a 10-task slow workflow (1s each). Immediately cancel. Confirm running tasks finish, rest CANCELLED.
- GET /workflows/:id after cancel — confirm workflow status is CANCELLED and task statuses are correct
- Try cancelling the same workflow again — confirm 200 idempotent no-op

Phase 8: Integration Tests (Deliverables)

8.1 What we build

This phase builds the four mandatory integration tests from the assignment deliverables. No new features — just comprehensive tests proving correctness of everything built so far.

8.2 Assignment boxes ticked

DELIVERABLE	500-task diamond DAG executing correctly under 10 concurrent workers
DELIVERABLE	kill -9 mid-execution test proving recovery
DELIVERABLE	Retry storm test (100 tasks each failing 5 times)
DELIVERABLE	Cron parser tests covering 15+ expressions including 0 0 29 2 *

8.3 Prompt for Claude Code

```
Build the four mandatory integration tests from the assignment deliverables.

1. 500-TASK DIAMOND DAG (tests/integration/diamond-dag.test.ts):
- Generate a 500-task diamond DAG programmatically:
- Node 0 is the source (no deps).
- Nodes 1-498 depend on node 0 (fan-out).
- Node 499 depends on all of nodes 1-498 (fan-in).
- All use 'noop' handler.
- Submit via POST /workflows.
- Wait for workflow to reach COMPLETED (poll with timeout).
- Assert: all 500 tasks COMPLETED.
- Assert: node 0 completed before any of 1-498.
- Assert: node 499 completed last.
- Assert: max concurrent running tasks never exceeded WORKER_COUNT (10).
- Use WORKER_COUNT=10 in this test.
- This test MUST complete in < 30 seconds.

2. KILL -9 RECOVERY TEST (tests/integration/kill-9.test.ts):
- Already scaffolded in Phase 5. Ensure it is robust:
- Spawn app as child process with specific PORT to avoid conflicts.
- Submit a 10-task chain with 'slow' handler (500ms each).
- Poll until 3 tasks are RUNNING.
- Send SIGKILL to child process.
- Wait 2 seconds.
- Start a new instance on the same port.
- Poll until workflow is terminal (timeout: 60s).
- Assert: all tasks are COMPLETED or FAILED. No RUNNING tasks remain.
- Assert: no task was executed more times than max_attempts.

3. RETRY STORM (tests/integration/retry-storm.test.ts):
- Register a 'fail-n-times' handler that fails the first N attempts
and succeeds on attempt N+1.
- Create a workflow with 100 independent tasks (no dependencies),
each using 'fail-n-times' with input { failCount: 5 },
retryPolicy: { maxAttempts: 6, backoffBase: 100, backoffCap: 1000 }.
```

- Submit and wait for COMPLETED.
- Assert: all 100 tasks COMPLETED with attempts=6.
- Assert: total execution count (across all retries) = 100 * 6 = 600.
- Assert: scheduled_at timestamps show decorrelated spread (not clustered).

Compute std deviation of scheduled_at across same-attempt-number groups.
It should be > 0 (jitter is present).

4. CRON PARSER TESTS – already built in Phase 6. Verify 15+ expressions pass.
If any missing, add them here.

UTILITY: Create a test helper (tests/helpers/test-server.ts) that:

- Starts the app on a random port.
- Provides HTTP client methods (submit workflow, poll status, cancel, etc.).
- Cleans up DB between tests (TRUNCATE workflows, tasks, task_dependencies, schedules).
- Provides waitForWorkflowStatus(workflowId, status, timeout) helper.

Run all integration tests sequentially (not parallel) to avoid port/DB conflicts.
Set test timeout to 120 seconds for the longer tests.

8.4 Manual verification

- Run **npm run test:integration** — all 4 suites green
- Check test durations: diamond DAG < 30s, retry storm < 60s
- Run **kill -9 test** 3 times: should pass consistently (no flakes)
- For retry storm: manually inspect scheduled_at timestamps in DB to see jitter spread

Phase 9: Documentation + Polish + Dockerfile

9.1 What we build

- **Design doc:** up to 3 pages covering data structure choices, trade-offs, production hardening
- **Architecture diagram:** scheduler, queue, worker pool, persistence, recovery loop (ASCII or Mermaid)
- **README.md:** setup instructions, one-command start, API reference, project structure
- **Dockerfile:** production-grade multi-stage build, non-root user, health check
- **docker compose up:** single command to start everything
- **Code polish:** remove TODOs, add missing JSDoc, ensure eslint clean, type-check clean

9.2 Assignment boxes ticked

DOCS 10%	Design doc: data structures, trade-offs, production hardening
DOCS 10%	Architecture diagram showing all subsystems
CODE QUALITY 15%	Clean TypeScript, no lint errors, consistent patterns
INFRA	Dockerfile + docker compose + one-command setup

9.3 Prompt for Claude Code

Final polish: documentation, architecture diagram, and production-grade Dockerfile.

DESIGN DOC (docs/design.md – up to 3 pages):

- Section 1: Data structures – min-heap ($O(\log n)$, compound key), bounded semaphore, DFS three-coloring. Why each was chosen over alternatives.
- Section 2: Concurrency & durability – strict 2-writes, atomic ready-set, heartbeat/reaper, decorrelated jitter. Why each approach.
- Section 3: Cron – timezone-aware, DST gap/fold handling, brute-force vs field-skipping.
- Section 4: Production hardening – multi-process with SKIP LOCKED, observability, named templates, rate limiting. What we'd add if shipping to production.

ARCHITECTURE DIAGRAM:

- ASCII diagram in the design doc showing: HTTP API -> Workflow Service -> Postgres, Scheduler Loop -> Min-Heap -> Worker Pool -> Handler, Reaper -> Postgres, Cron Scheduler -> Workflow Service.

README.md:

- Quick start: `docker compose up --build`
- API reference: `POST /workflows`, `GET /workflows/:id`, `POST /workflows/:id/cancel`, `POST /schedules`, `GET /health`.
- Project structure overview.
- How to run tests: `npm test (unit)`, `npm run test:integration`.
- Design decisions: link to `docs/design.md`.

DOCKERFILE:

- Multi-stage: builder (`npm ci + tsup build`) + runner (`node:20-alpine`, copy dist).
- Non-root user. `HEALTHCHECK` CMD.
- `docker-compose.yml`: volumes for postgres data, healthcheck for postgres.

CODE POLISH:

- `npm run lint` — 0 errors.
- `npm run typecheck (tsc --noEmit)` — 0 errors.
- Remove all `TODO/FIXME` comments.
- JSDoc on all public methods of core classes.
- Ensure all test files import from `src/`, not from `dist/`.

PACKAGE.JSON SCRIPTS:

- `dev`: `tsx watch src/index.ts`
- `build`: `tsup src/index.ts`
- `start`: `node dist/index.js`
- `test`: `vitest run`
- `test:integration`: `vitest run tests/integration/ --pool=forks --poolOptions.forks.singleFork`
- `lint`: `eslint src/ tests/`
- `typecheck`: `tsc --noEmit`
- `migrate`: `tsx src/db/migrate.ts`

9.4 Manual verification (final checklist)

- **docker compose down -v && docker compose up --build** — starts from scratch, app healthy
- **npm run lint** — 0 errors
- **npm run typecheck** — 0 errors
- **npm test** — all unit tests green
- **npm run test:integration** — all integration tests green (diamond DAG, kill -9, retry storm, cron)
- Read `docs/design.md` — coherent, <= 3 pages, covers trade-offs and production hardening
- Read `README.md` — clear setup instructions, API reference, project structure
- Submit a fresh workflow from scratch on a clean docker compose — full end-to-end works
- Mentally rehearse: can you walk through every file and defend every design choice?

Appendix: Rubric Cross-Reference

Criterion	Weight	Covered by
Correctness under edge cases	30%	Phase 1 (cycle detect), Phase 3 (ready-set), Phase 4 (retry), Phase 6 (cron DST), Phase 8 (all deliverables)
Data structure design + quality	25%	Phase 1 (heap, semaphore, DFS), Phase 3 (scheduler), Phase 6 (cron parser)
Concurrency, durability, crash recovery	20%	Phase 3 (atomic decrement), Phase 5 (2-writes, reaper, kill -9), Phase 7 (cancel drain)
Code quality, typing, tests	15%	Phase 0 (strict TS, ESLint, Vitest), all phases (unit + integration tests)

Criterion	Weight	Covered by
Documentation + trade-off defence	10%	Phase 9 (design doc, architecture diagram, README)